UNIVERSITY COLLEGE OF ENGINEERING

NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM,NAGERCOIL-629 004

RECORD NOTE BOOK

CCS365– SOFTWARE DEFINED NETWORKS

Register No :

Name :

Year/Semester :

Department :

UNIVERSITY COLLEGE OF ENGINEERING

NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM,NAGERCOIL-629 004

Register No:

Certified that, this is the bonafide record of work done by ***Mr./Ms.*** *…… of VI Semester in Computer Science and Engineering of this college, in the CCS365 -Software Defined Networks during the academic year 2025-2026 in partial fulfilment of the requirements of the B.E Degree course of the Anna University Chennai.*

Staff-in-charge **Head of the Department**

This record is submitted for the University Practical Examination held on

……

Internal Examiner **External Examiner**

LIST OF EXPERIMENTS

Exp No	Date	Title	Page	Sign
1.		**Setup your own virtual SDN lab** **1a. Virtual box mininet environment for** **SDN http.fifimininet.org** **1b. https:fifiwww.kathara.org** **1c. GNS3**		
2.		**Create a simple mininet topology with SDN Controller and use Wireshark to capture and** **Visualize the OpenFlow messages such as OpenFlow FLOW_MOD, PACKET_IN, PACKET_OUT etc.**		
3.		**Create a SDN application that uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall etc..**		
4.		**Create a simple end-to-end network service with two VNFs using vim-emu.**		
5.		**Install OSM and onboard and orchestrate network service.**		

Ex No : 1(a) Date :	Set Up Your Own Virtual SDN Lab Using Mininet Environment

AIM:

To set up a virtual Software Defined Networking (SDN) lab using the Mininet environment on Ubuntu, allowing the creation and testing of custom virtual network topologies.

PROCEDURE:

1.Update the Package Repository:

This command updates the local package list from the Ubuntu repositories. It is used to ensure the latest information about software packages and versions is available before installation. This is the first step to the setting up of the mininet environment.

sudo apt update

```
cse@ubuntu:~$ sudo apt update
[sudo] password for cse:
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
79 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

2.Upgrade Installed Packages:

Upgrades all installed packages to their latest available versions. This commands helps us to make sure the system is up-to-date, stable, and compatible with the Mininet installation. This is the next step in setting up the mininet environment . this command helps in fetching the package information this checks which of your installed packages have new versions available. It then downloads and installs the updates for those packages keeping your system up-to-date. It doesnot remove or install new packages it only upgrades what's already installed. It may prompt you to approve the upgrade process if it involves a lot changes or disk space.

sudo apt upgrade

```
cse@ubuntu:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
The following package was automatically installed and is no longer required:
  gir1.2-goa-1.0
Use 'sudo apt autoremove' to remove it.
The following NEW packages will be installed:
  libxmlb2 ubuntu-advantage-desktop-daemon ubuntu-pro-client
  ubuntu-pro-client-l10n
The following packages will be upgraded:
  apport apport-gtk apt apt-utils base-files bolt distro-info distro-info-data
  dns-root-data fwupd fwupd-signed gir1.2-notify-0.7 iptables iputils-ping
  iputils-tracepath isc-dhcp-client isc-dhcp-common libapt-pkg6.0 libcom-err2
  libfprint-2-2 libfprint-2-tod1 libfwupd2 libfwupdplugin5 libgpgme11
  libip4tc2 libip6tc2 libnl-3-200 libnl-genl-3-200 libnl-route-3-200
  libnotify-bin libnotify4 libnss-systemd libpam-systemd libpcap0.8
  libsmbclient libsnmp-base libsnmp35 libss2 libsystemd0 libunwind8
  libwbclient0 libxtables12 linux-firmware logsave ltrace python-apt-common
  python3-apport python3-apt python3-debian python3-distro-info
  python3-distupgrade python3-problem-report python3-software-properties
  python3-tz python3-update-manager samba-libs snapd
  software-properties-common software-properties-gtk systemd systemd-sysv
  systemd-timesyncd tcpdump thermald tracker-extract tracker-miner-fs
  ubuntu-advantage-tools ubuntu-drivers-common ubuntu-release-upgrader-core
  ubuntu-release-upgrader-gtk ufw unzip update-manager update-manager-core
  update-notifier update-notifier-common wireless-regdb xdg-desktop-portal
  xserver-xorg-video-amdgpu
79 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 184 MB of archives.
After this operation, 69.3 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 base-files amd64 11ubuntu5.8 [60.3 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libnss-systemd amd64 245.4-4ubuntu3.24 [95.8 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 systemd-timesyncd amd64 245.4-4ubuntu3.24 [28.1 kB
```

3. Install Mininet:

Installs Mininet along with all necessary dependencies. Mininet is the tool used to emulate SDN networks. It allows creation of virtual hosts, switches, controllers, and links. This helps in installing the mininet into the ubuntu and helps us to easily use it via the terminal.

sudo apt-get install mininet

```
cse@ubuntu:~$ sudo apt install mininet
Reading package lists... Done
Building dependency tree
Reading state information... Done
mininet is already the newest version (2.2.2-5ubuntu1).
The following package was automatically installed and is no longer required:
  gir1.2-goa-1.0
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
```

4. Verify Mininet Installation:

Displays the version of the Mininet installed. This command help us to confirm successful installation and verify the version compatibility. Using this command we get clear idea on which version of the mininet we are using and helps us in making decision whether we need to install any other version .

mn --version

```
cse@ubuntu:~$ mn --version
2.3.1b4
```

5. Test Mininet:

With Default Topology Starts a Mininet session using Open vSwitch (ovsk) and performs a ping test between all nodes. This verifies whether the virtual network (with switches and hosts) is functioning correctly and that the hosts can communicate. This command will create a mininet network with open vSwitch bridges.

sudo mn --switch ovsbr --test pingall

```
cse@ubuntu:~$ sudo mn --switch ovsbr --test pingall
[sudo] password for cse:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 0.287 seconds
cse@ubuntu:~$ mn --version
2.3.1b4
cse@ubuntu:~$
```

6. Install Open vSwitch:

Installs the Open vSwitch software.OVS is a production-quality multilayer virtual switch used to manage traffic between virtual machines and integrate with

SDN controllers. This command installs the open vswitch which is used to create and manage virtual switches.

sudo apt-get install openvswitch-switch

```
cse@ubuntu:~$ sudo apt-get install openvswitch-switch
Reading package lists... Done
Building dependency tree
Reading state information... Done
openvswitch-switch is already the newest version (2.13.8-0ubuntu1.4).
The following package was automatically installed and is no longer required:
  gir1.2-goa-1.0
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

7. Start Mininet in Interactive Mode :

Launches Mininet in command-line interface mode. This command helps to allow custom creation and manipulation of network topologies by entering interactive commands. We start the mininet virtual machine by this command.

sudo mn

```
cse@ubuntu:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

8. Explore the Virtual Network in Mininet CLI:

Inside the Mininet CLI, you can use the following commands:

a)View Available Nodes:

Displays all current nodes in the network.

nodes

```
*** Starting CLI:
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet> net
```

b)View Network Links:

Displays the connectivity between nodes (i.e., links between hosts and switches).

net

```
c0 h1 h2 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet> dump
```

c)View Network Details:

Provides detailed information about each node, including interfaces, IP addresses, and PIDs.

dump

```
c0
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=51428>
<Host h2: h2-eth0:10.0.0.2 pid=51430>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=51435>
<Controller c0: 127.0.0.1:6653 pid=51421>
mininet>
```

RESULT:

Thus, the virtual SDN lab was successfully set up using the Mininet environment. All components (hosts, switches, controller) were created and verified using built-in Mininet tests.

Ex No :1(b) Date :	To Set up a Virtual SDN Lab by Installing Kathara

AIM:

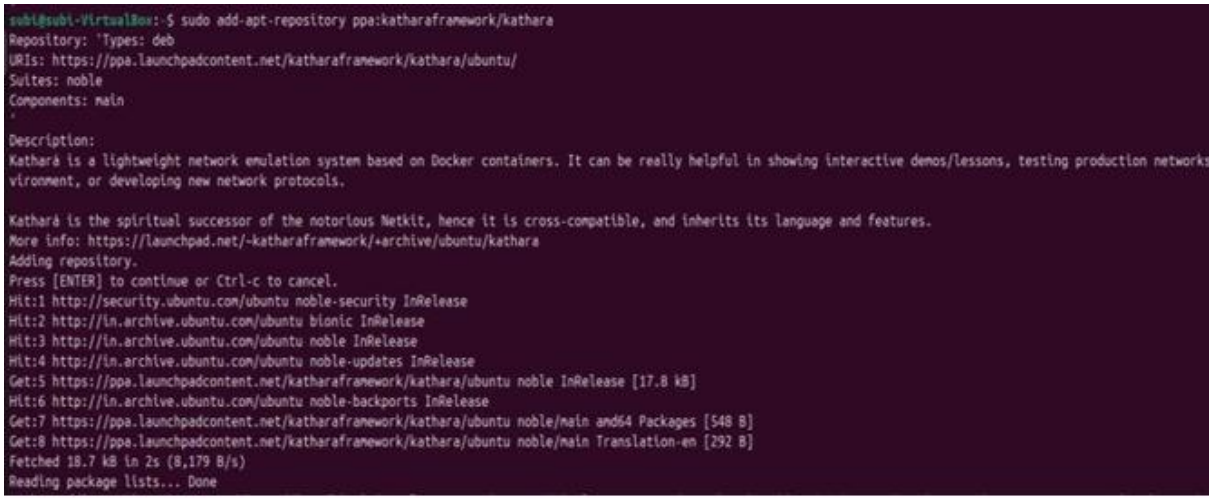
To set up a virtual SDN Lab by Installing kathara.

PROCEDURE:

Step 1:

A repository is like a software store for Linux. It contains packages (software) that your system can install using commands like apt. Without adding this repository, your system won't know how to find the Kathara package.

$sudo add-apt-repository ppa:katharaframework/kathara



Step 2:

Updates your system's knowledge of available software packages from all added repositories. After adding a new repository your system doesn't yet know what packages are available from it. This command updates the list so that you can install Kathara.

$sudo apt update

```
subi@subi-VirtualBox:~$ sudo apt update
Hit:1 http://in.archive.ubuntu.com/ubuntu bionic InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 https://ppa.launchpadcontent.net/katharaframework/kathara/ubuntu noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
148 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Step 3:

Downloads and installs the Kathara framework on your machine. Kathara is a container-based network emulator, used for creating and testing virtual network topologies.

$sudo apt install kathara

```
subi@subi-VirtualBox:~$ sudo apt install kathara
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-tools-6.8.0-39 linux-tools-6.8.0-39-generic
Use 'sudo apt autoremove' to remove them.
Suggested packages:
  docker.io | docker-ce
The following NEW packages will be installed:
  kathara
0 upgraded, 1 newly installed, 0 to remove and 83 not upgraded.
Need to get 30.6 MB of archives.
After this operation, 154 MB of additional disk space will be used.
Get:1 https://ppa.launchpadcontent.net/katharaframework/kathara/ubuntu noble/main amd64 kathara amd64 3.7.9-1noble [30.6 MB]
Fetched 30.6 MB in 1min 46s (289 kB/s)
Selecting previously unselected package kathara.
(Reading database ... 238548 files and directories currently installed.)
Preparing to unpack .../kathara_3.7.9-1noble_amd64.deb ...
Unpacking kathara (3.7.9-1noble) ...
Setting up kathara (3.7.9-1noble) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
```

Step 4:

Docker is a platform used to run applications in isolated environments called containers. Kathara uses containers to simulate each network node (like a router or switch). Docker handles the creation and management of these containers. Kathara uses containers to simulate each network node (like a router or switch). Docker handles the creation and management of these containers.

$sudo apt install docker.io

```
subi@subi-VirtualBox:~$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-tools-6.8.0-39 linux-tools-6.8.0-39-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  bridge-utils containerd pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd docker.io pigz runc ubuntu-fan
0 upgraded, 6 newly installed, 0 to remove and 83 not upgraded.
Need to get 78.2 MB of archives.
After this operation, 301 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.1.12-0ubuntu3.1 [8,599 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.24-0ubuntu1-24.04.2 [37.0 MB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 26.1.3-0ubuntu1-24.04.1 [32.4 MB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 ubuntu-fan all 0.12.16 [35.2 kB]
Fetched 78.2 MB in 22s (3,580 kB/s)
Preconfiguring packages ...
Selecting previously unselected package pigz.
(Reading database ... 238623 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.8-1_amd64.deb ...
Unpacking pigz (2.8-1) ...
```

Step 5:

A lightweight terminal emulator for the X Window System (used in Linux graphical environments). When you run a virtual node in Kathara, xterm opens a terminal window so you can interact with that node.

$sudo apt install xterm

```
subi@subi-VirtualBox:~$ sudo apt install xterm
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step 6:

Check Kathara version. Confirms successful installation by checking the installed version.

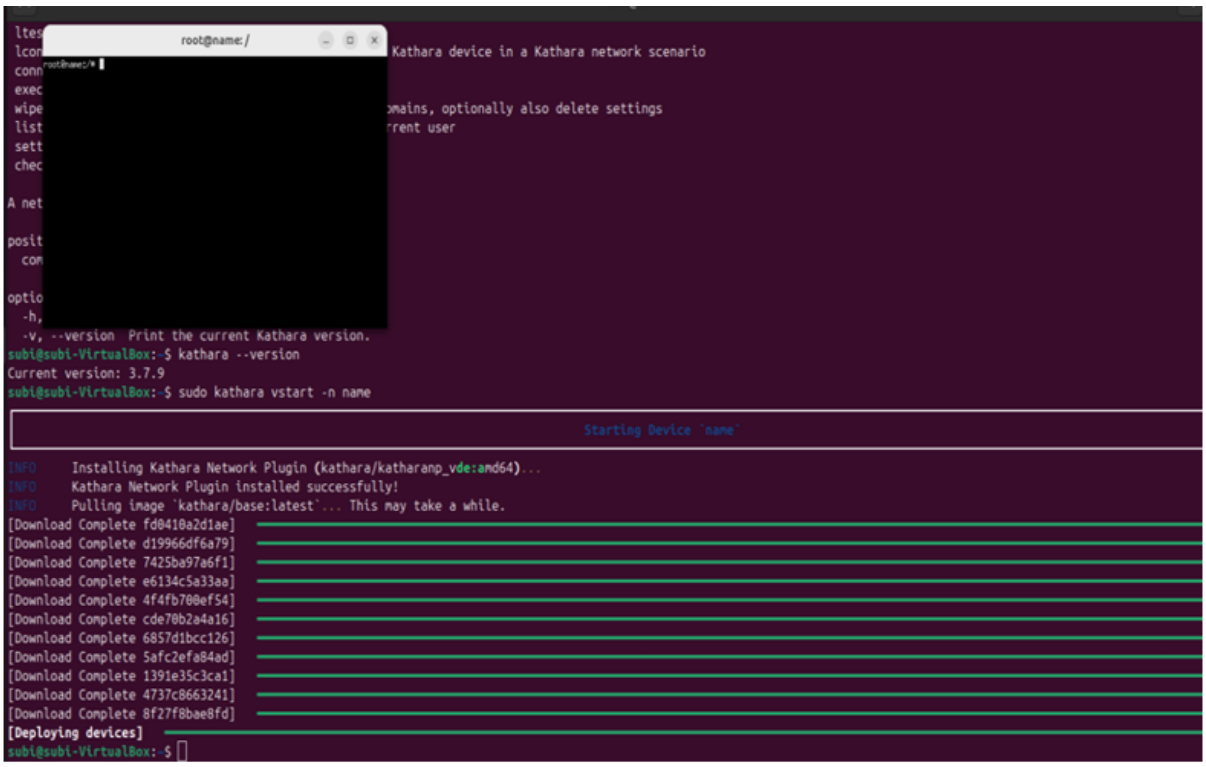
$kathara --version

```
subi@subi-VirtualBox:~$ kathara --version
Current version: 3.7.9
```

Step 7:

Start Kathara. Runs a sample virtual network to verify installation.

$sudo kathara vstart -n name



RESULT:

Thus, the virtual box environment for SDN Lab using kathara was completed successfully.

EX NO : 1(c) Date :	Setup your own virtual SDN Lab using GNS3

AIM :

To set up a virtual SDN Lab by Installing GNS3.

PROCEDURE :

Step 1: Add the gns3 repository to your repository.

To begin setting up a virtual Software Defined Networking (SDN) lab using GNS3, the first step involves adding the official GNS3 repository to your system's package manager. This ensures that you can install the latest stable version of GNS3 and receive regular updates.

$ sudo add-apt-repository ppa:gns3/ppa

- Add the GNS3 PPA to your list of APT sources.
- Ensures access to the latest GNS3 packages.
- Enables you to install and update GNS3 using apt

```
cse@ubuntu:~$ sudo add-apt-repository ppa:gns3/ppa
[sudo] password for cse:
 PPA for GNS3 and Supporting Packages. Please see http://www.gns3.com for more
details
 More info: https://launchpad.net/~gns3/+archive/ubuntu/ppa
Press [ENTER] to continue or Ctrl-c to cancel adding it.

Get:1 http://ppa.launchpad.net/gns3/ppa/ubuntu focal InRelease [18.3 kB]
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:5 http://ppa.launchpad.net/katharaframework/kathara/ubuntu focal InRelease
Hit:6 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Get:7 http://ppa.launchpad.net/gns3/ppa/ubuntu focal/main amd64 Packages [1,832
 B]
Get:8 http://ppa.launchpad.net/gns3/ppa/ubuntu focal/main Translation-en [996 B
]
Fetched 21.1 kB in 1s (17.8 kB/s)
Reading package lists... Done
```

Step 2: Update the version installed.

After adding the GNS3 repository, the next step is to update the system's package list to include the newly added PPA. This is done using the command sudo apt update, which refreshes the local cache of available packages and their versions. This ensures that your system recognizes the latest GNS3 packages available for installation.

$ sudo apt update

```
cse@ubuntu:~$  sudo apt update
Hit:1 http://ppa.launchpad.net/gns3/ppa/ubuntu focal InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:4 http://ppa.launchpad.net/katharaframework/kathara/ubuntu focal InRelease
Hit:5 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:6 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Step 3: Install GNS3

With the repository added and the package list updated, you can now install GNS3 by running the command:

$ sudo apt install gns3-gui gns3-server

This command installs both the graphical user interface (gns3-gui) and the backend server component (gns3-server) required for GNS3 to function. The GUI provides a user-friendly environment to design and manage network topologies, while the server handles device emulation and simulation tasks in the background.

Step 4: Install Docker CE

To enable container-based network simulation in your GNS3 SDN lab, you need to install Docker Community Edition (CE). This is done using the command:

$ sudo apt install docker-ce

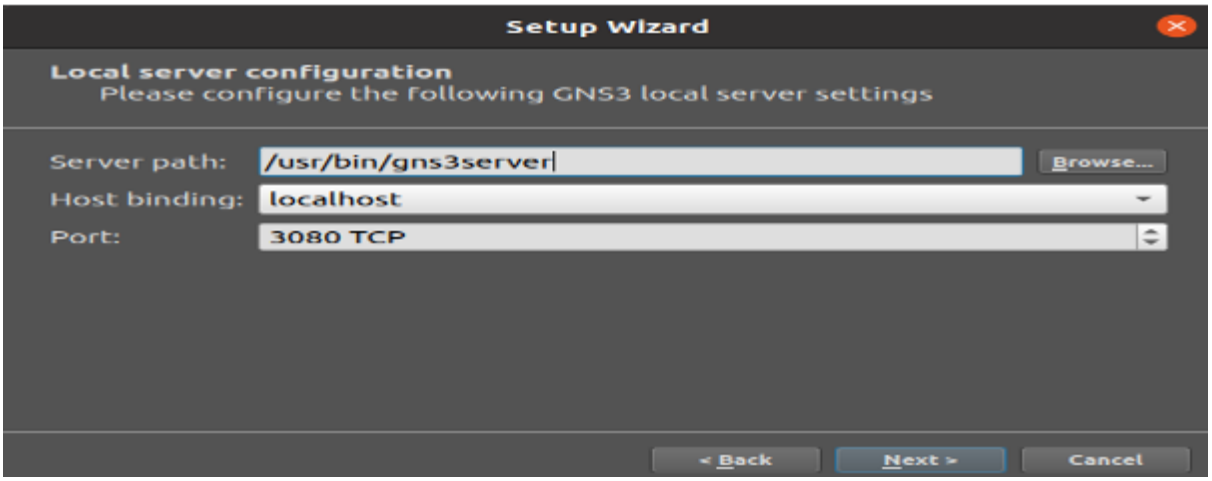
Step 5: Start GNS3

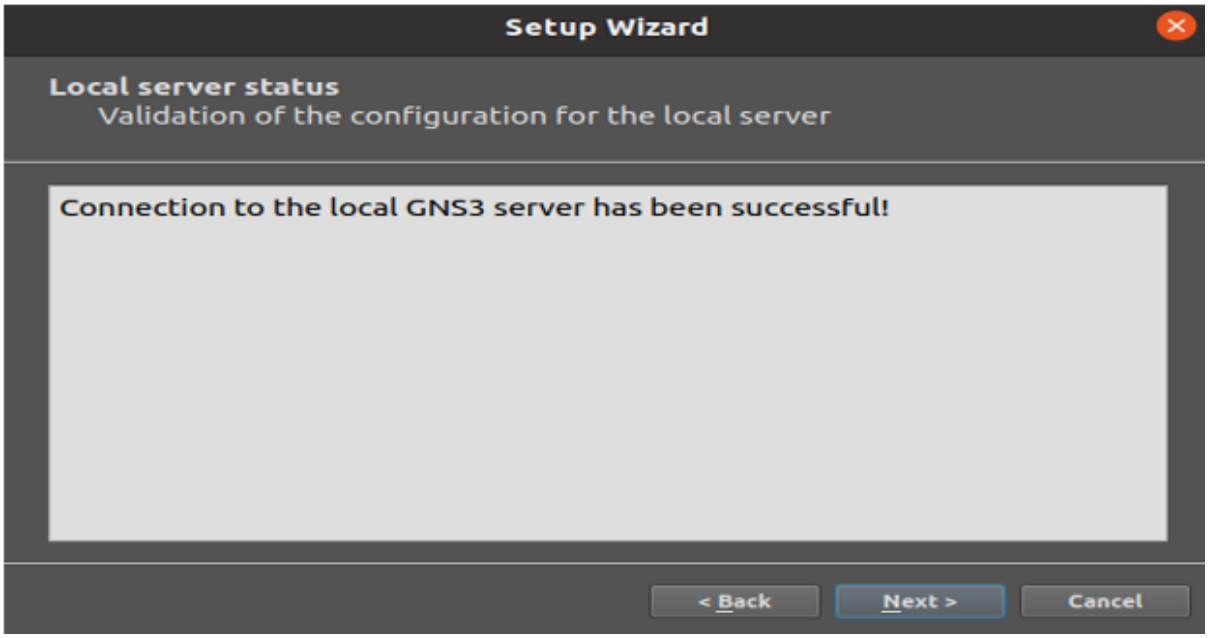
Once all required components, including Docker, are installed, you can launch the GNS3 application using the following command:

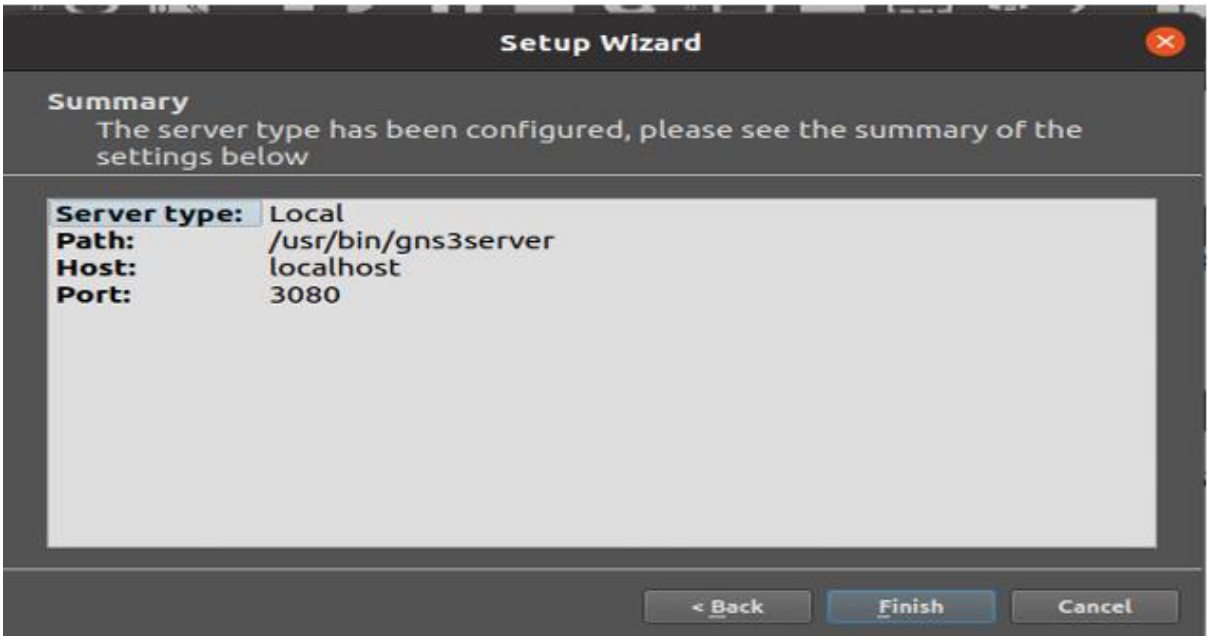
$ gns3

This command opens the GNS3 graphical user interface, where you can begin building and simulating network topologies. On first launch, GNS3 may prompt you to configure a local or remote GNS3 server, as well as optional settings like enabling Docker support or integrating VirtualBox/VMware. After configuration, you'll see the main workspace where network devices can be dragged, connected, and simulated—signaling that your SDN lab setup is complete and ready for use.

```
cse@ubuntu:~$ gns3
2025-03-12 00:00:43 INFO root:126 Log level: INFO
2025-03-12 00:00:43 INFO main:265 GNS3 GUI version 2.2.53
2025-03-12 00:00:43 INFO main:266 Copyright (c) 2007-2025 GNS3 Technologies Inc
.
2025-03-12 00:00:43 INFO main:267 Application started with /usr/bin/gns3
2025-03-12 00:00:55 INFO controller:437 Listening for controller notifications
on 'ws://localhost:3080/v2/notifications/ws'
2025-03-12 00:00:55 INFO controller:437 Listening for controller notifications
on 'ws://localhost:3080/v2/notifications/ws'
```







RESULT:

Thus, the virtual box environment for SDN Lab using GNS3 was completed Successfully.

Ex No : 2 Date :	Creation of a simple mininet topology with SDN Controller and visualuzation of openflow messages Wireshark

AIM:

1. To create a simple Mininet network with an SDN controller.
2. To capture OpenFlow messages such as FLOW_MOD, PACKET_IN, PACKET_OUT using Wireshark.
3. To visualize the captured OpenFlow messages for analysis.

PROCEDURE:

Step 1 : Install Mininet

Open terminal and execute:

$ sudo apt-get update

```
jothi@ubuntu:~$ sudo apt-get update
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
```

$ sudo apt-get install mininet

```
jothi@ubuntu:~$ sudo apt-get install mininet
Reading package lists... Done
Building dependency tree
Reading state information... Done
mininet is already the newest version (2.2.2-5ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 402 not upgraded.
```

Step 2: Install POX Controller

Install the pox controller into your system.

$ git clone https://github.com/noxrepo/pox.git

```
jothi@ubuntu:~$ git clone https://github.com/noxrepo/pox.git
Cloning into 'pox'...
remote: Enumerating objects: 13425, done.
remote: Counting objects: 100% (277/277), done.
remote: Compressing objects: 100% (50/50), done.
remote: Total 13425 (delta 258), reused 227 (delta 227), pack-reused 13148 (from 2)
Receiving objects: 100% (13425/13425), 5.14 MiB | 3.54 MiB/s, done.
Resolving deltas: 100% (8704/8704), done.
```

Step 3: Install Wireshark

Install wireshark using the following command.

$ sudo apt-get install wireshark

```
jothi@ubuntu:~$ sudo apt-get install wireshark
Reading package lists... Done
Building dependency tree
Reading state information... Done
wireshark is already the newest version (3.2.3-1).
0 upgraded, 0 newly installed, 0 to remove and 402 not upgraded.
```

Step 4: Start POX Controller

Navigate to the POX directory and start the POX controller.You can use the 'pox.py' script along with a specific module.For example,you can use the 'forwarding.l2_learning' module.

$ cd ~/pox

```
jothi@ubuntu:~$ cd ~/pox
```

$ ./pox.py forwarding.l2_learning

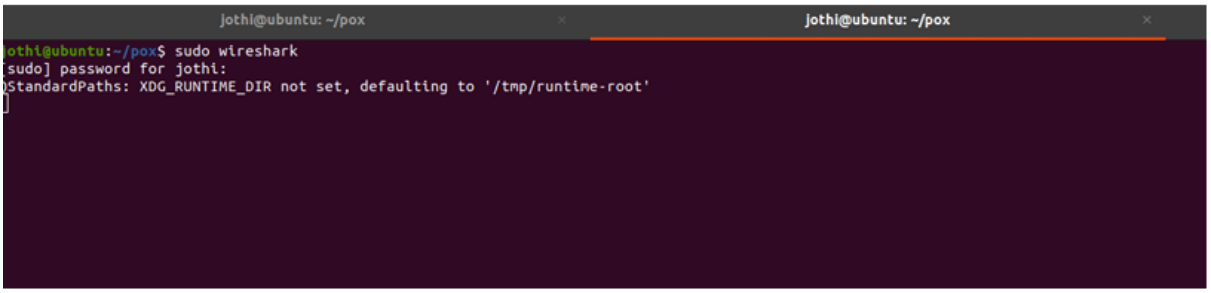
```
jothi@ubuntu:~/pox$ ./pox.py forwarding.l2_learning
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
```

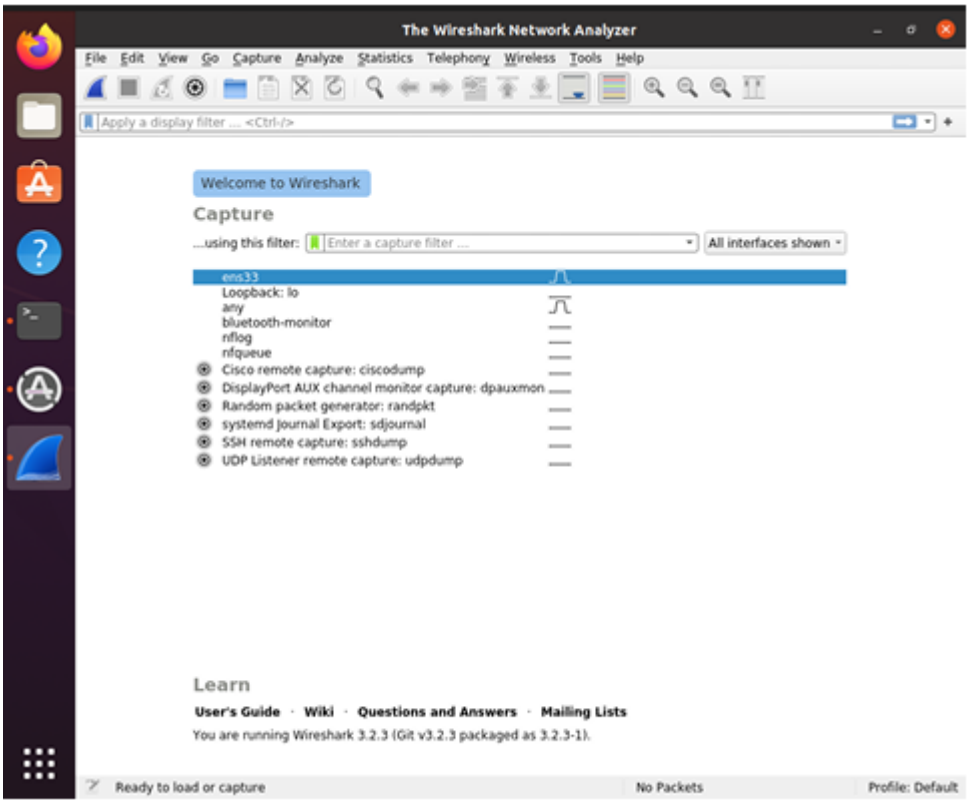
Keep this terminal running.

Step 5: Start Wireshark

Open a new terminal:

$ sudo wireshark



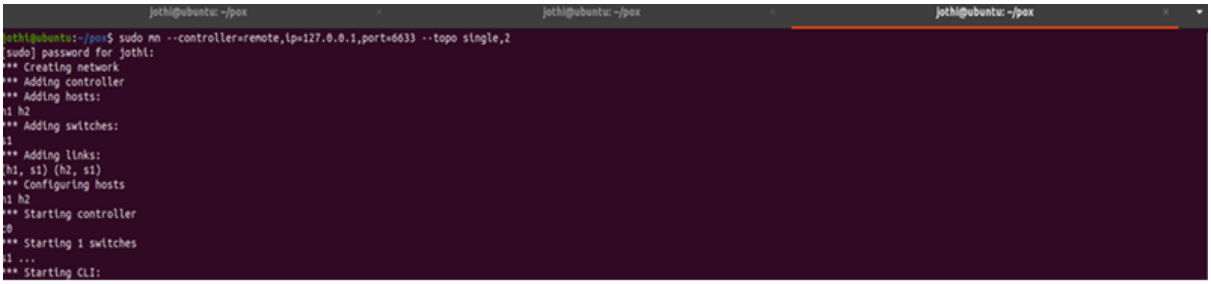


- Select Loopback (lo)
- Click Start Capture

Step 6: Start Mininet Topology

Open another new terminal:

$ sudo mn --controller=remote,ip=127.0.0.1,port=6633 --topo single,2



This creates:

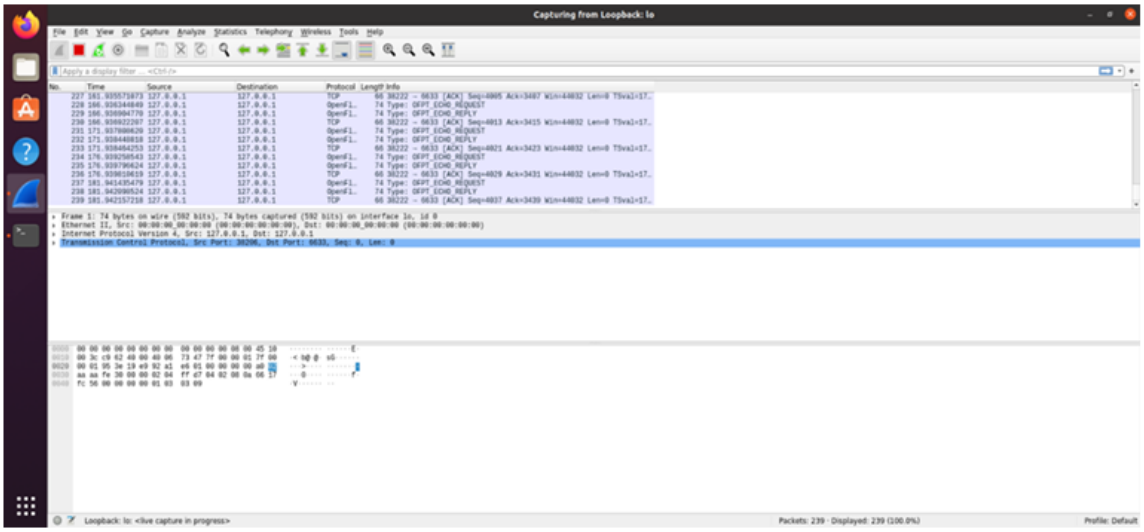
- 1 Switch
- 2 hosts

Step 7: Verify Network Connectivity

In mininet CLI:

```
$ pingall
```

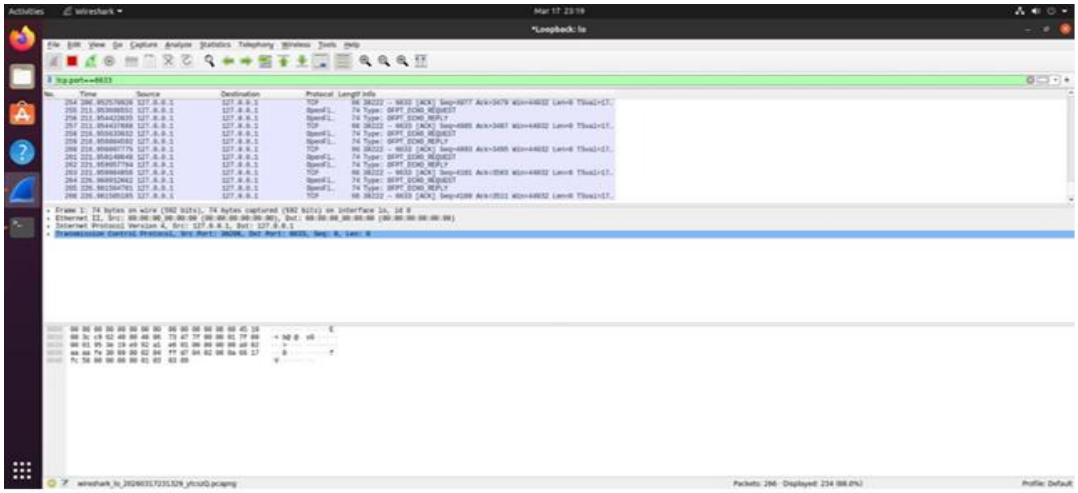
```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```



Step 8: Capture OpenFlow Messages

In Wireshark, apply filter:

```
$ tcp.port == 6633
```



Step 9:Stop Wireshark Capture

Stop capturing packets in Wireshark when you've generated enough OpenFlow messages.

RESULT:

Thus, a simple mininet topology with SDN Controller was created , and to capture and visualize the openflow messages such as Openflow FLOW_MODE, PACKET_IN, PACKET_OUT etc using wireshark was executed successfully.

Ex No : 3 Date :	**Create a SDN application that used the Northbound API Program flow table rules on the switch for various Use cases like L2 learning switch, Traffic Engineering, firewall etc.**

AIM:

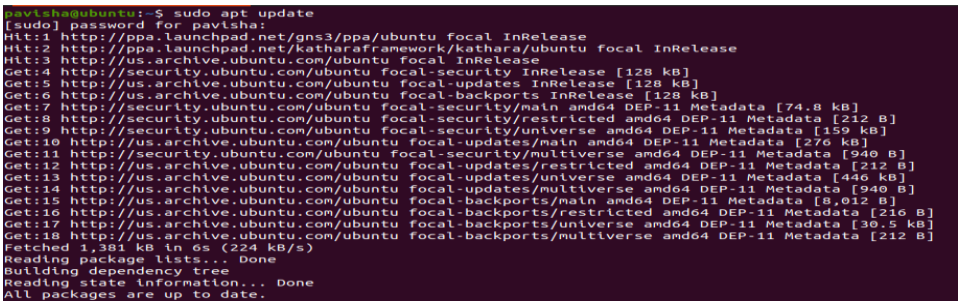
Develop an SDN application that uses the Northbound API to program flow table rules on SDN switches for different use cases: L2 learning switch, Traffic Engineering, and Firewall.

PROCEDURE:

1.Install Required Packages:

Install the necessary networking tools such as Mininet, Open vSwitch, and Python package manager using the following command:

```
sudo apt update
sudo apt install mininet openvswitch-switch python3-pip -y
```



2.Install Ryu Controller:

Install the Ryu SDN controller framework which will be used to control the network behavior.

```
sudo apt install python3-ryu -y
```

```
pavisha@ubuntu:~$ sudo apt install python3-ryu -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following package was automatically installed and is no longer required:
  gir1.2-goa-1.0
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  docutils-common ieee-data javascript-common libjs-jquery libjs-sphinxdoc libjs-underscore
  python-babel-localedata python3-babel python3-bcrypt python3-bs4 python3-debtcollector
  python3-dnspython python3-docutils python3-eventlet python3-greenlet python3-html5lib
  python3-iso8601 python3-lxml python3-monotonic python3-msgpack python3-netaddr
  python3-oslo.config python3-oslo.context python3-oslo.i18n python3-oslo.log
  python3-oslo.serialization python3-oslo.utils python3-paramiko python3-pbr python3-pygments
  python3-pyinotify python3-pyparsing python3-repoze.lru python3-rfc3986 python3-roman
  python3-routes python3-soupsieve python3-stevedore python3-tinyrpc python3-webencodings
  python3-webob python3-wrapt
Suggested packages:
  apache2 | lighttpd | httpd python-debtcollector-doc docutils-doc fonts-linuxlibertine
  | ttf-linux-libertine texlive-lang-french texlive-latex-base texlive-latex-recommended
  python-eventlet-doc python-greenlet-doc python-greenlet-dev python3-greenlet-dbg
  python3-genshi python3-lxml-dbg python-lxml-doc ipython3 python-netaddr-docs
  python-oslo.log-doc python3-gssapi python-pygments-doc ttf-bitstream-vera
  python-pyinotify-doc python-pyparsing-doc python3-paste python-ryu-doc python-tinyrpc-doc
  python-webob-doc
The following NEW packages will be installed:
  docutils-common ieee-data javascript-common libjs-jquery libjs-sphinxdoc libjs-underscore
  python-babel-localedata python3-babel python3-bcrypt python3-bs4 python3-debtcollector
  python3-dnspython python3-docutils python3-eventlet python3-greenlet python3-html5lib
  python3-iso8601 python3-lxml python3-monotonic python3-msgpack python3-netaddr
  python3-oslo.config python3-oslo.context python3-oslo.i18n python3-oslo.log
  python3-oslo.serialization python3-oslo.utils python3-paramiko python3-pbr python3-pygments
  python3-pyinotify python3-pyparsing python3-repoze.lru python3-rfc3986 python3-roman
  python3-routes python3-ryu python3-soupsieve python3-stevedore python3-tinyrpc
  python3-webencodings python3-webob python3-wrapt
0 upgraded, 43 newly installed, 0 to remove and 0 not upgraded.
Need to get 11.6 MB of archives.
After this operation, 63.7 MB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal/main amd64 docutils-common all 0.16+dfsg-2 [116 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal/main amd64 ieee-data all 20180805.1 [1,589 kB]
```

Ryu acts as the central controller that communicates with network switches.

3. Verify Installation of Ryu:

Check whether the Ryu controller is installed correctly by executing:

ryu-manager --version

```
pavisha@ubuntu:~$  ryu-manager --version
ryu-manager 4.30
pavisha@ubuntu:~$
```

4. Create SDN Application File:

Create a Python file to implement the SDN controller logic:

nanosdn_app.py

```
pavisha@ubuntu:~$ nano sdn_app.py
pavisha@ubuntu:~$
```

Program:

```
from ryu.base import app_manager

from ryu.controller import ofp_event
```

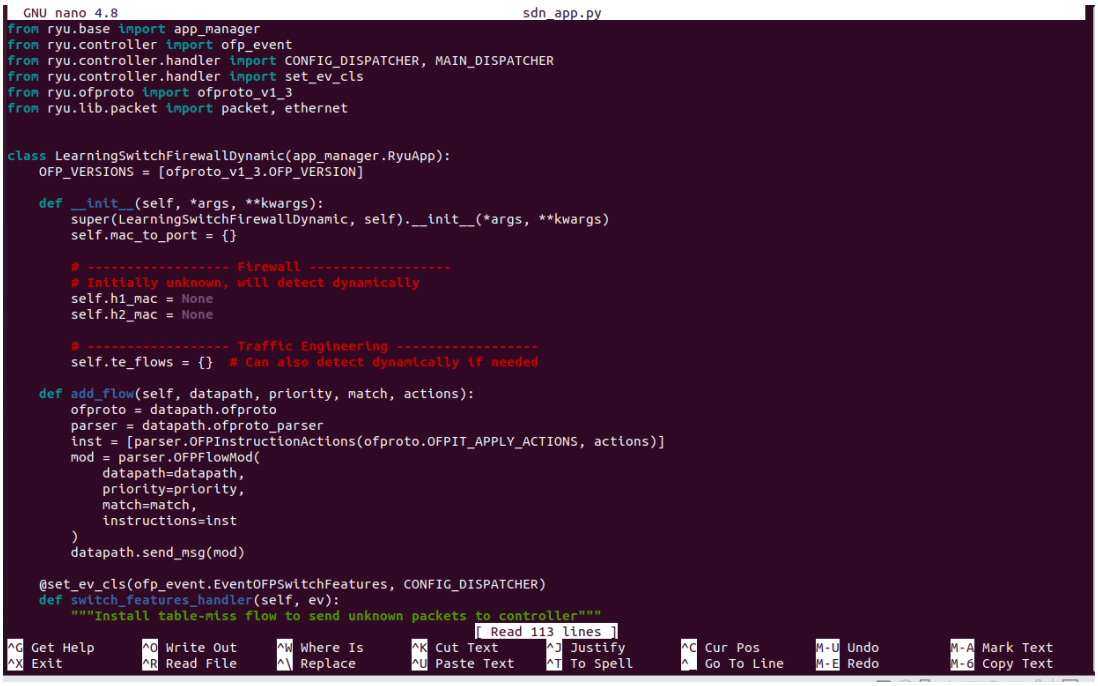
```
from ryu.controller.handler import CONFIG_DISPATCHER,
MAIN_DISPATCHER

from ryu.controller.handler import set_ev_cls

from ryu.ofproto import ofproto_v1_3

from ryu.lib.packet import packet, ethernet

class
LearningSwitchFirewallDynamic(app_manager.RyuApp):

    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):

        super(LearningSwitchFirewallDynamic,
self).__init__(*args, **kwargs)

        self.mac_to_port = {}

        # ------------------ Firewall ---------------
---

        # Initially unknown, will detect dynamically

        self.h1_mac = None

        self.h2_mac = None

        # ------------------ Traffic Engineering ----
--------------

        self.te_flows = {}  # Can also detect
dynamically if needed

        def add_flow(self, datapath, priority, match,
actions):

        ofproto = datapath.ofproto

        parser = datapath.ofproto_parser

        inst =
[parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACT
IONS, actions)]
```

```
        mod = parser.OFPFlowMod(
            datapath=datapath,
            priority=priority,
            match=match,
            instructions=inst
        )
        datapath.send_msg(mod)
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        """Install table-miss flow to send unknown
packets to controller"""
        datapath = ev.msg.datapath
        parser = datapath.ofproto_parser
        ofproto = datapath.ofproto
        match = parser.OFPMatch()
        actions =
[parser.OFPActionOutput(ofproto.OFPP_CONTROLLER)]
        self.add_flow(datapath, 0, match, actions)
    @set_ev_cls(ofp_event.EventOFPPacketIn,
MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        datapath = msg.datapath
        parser = datapath.ofproto_parser
        ofproto = datapath.ofproto
        in_port = msg.match['in_port']
```

```
        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocol(ethernet.ethernet)
        if eth is None:
            return
        src = eth.src
        dst = eth.dst
        dpid = datapath.id
        # Learn MAC addresses
        self.mac_to_port.setdefault(dpid, {})
        self.mac_to_port[dpid][src] = in_port
        # ------------------ Dynamic MAC detection ----------------
        if self.h1_mac is None:
            self.h1_mac = src
            self.logger.info(f"Detected h1 MAC: {self.h1_mac}")
        elif self.h2_mac is None and src != self.h1_mac:
            self.h2_mac = src
            self.logger.info(f"Detected h2 MAC: {self.h2_mac}")
        # ------------------ Firewall ----------------
        if self.h1_mac and self.h2_mac:
            if src == self.h1_mac and dst == self.h2_mac:
                self.logger.info(f"Firewall: Dropping {src} -> {dst}")
```

```
                match =
parser.OFPMatch(in_port=in_port, eth_src=src,
eth_dst=dst)
                self.add_flow(datapath, 30, match,
actions=[])  # Drop flow
                return  # Do not forward packet
        # ------------------ Traffic Engineering ----
--------------
        out_port = None
        for (t_src, t_dst), port in
self.te_flows.items():
            if src == t_src and dst == t_dst:
                out_port = port
        # Default Learning Switch behavior
        if out_port is None:
            if dst in self.mac_to_port[dpid]:
                out_port =
self.mac_to_port[dpid][dst]
            else:
                out_port = ofproto.OFPP_FLOOD
        actions = [parser.OFPActionOutput(out_port)]
        # Install flow for faster forwarding
        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port,
eth_dst=dst)
            priority = 10
            self.add_flow(datapath, priority, match,
actions)
```

```
# Send packet out
out = parser.OFPPacketOut(
    datapath=datapath,
    buffer_id=msg.buffer_id,
    in_port=in_port,
    actions=actions,
    data=msg.data
)
datapath.send_msg(out)
```



Save the file using:

- CTRL + O → Press Enter
- CTRL + X → Exit the editor

4. Clean Previous Mininet Configuration:

Before creating a new network, clear any existing Mininet topology:

sudo mn -c

```
pavisha@ubuntu:~$ sudo mn -c
[sudo] password for pavisha:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller udpbwtest mnexec ivs 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller udpbwtest mnexec ivs 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
```

This avoids conflicts with previously created virtual networks.

6. Start the Ryu Controller:

Run the Ryu controller using the created application file:

ryu-manager sdn_app.py

```
pavisha@ubuntu:~$ ryu-manager sdn_app.py
loading app sdn_app.py
loading app ryu.controller.ofp_handler
instantiating app sdn_app.py of LearningSwitchFirewallDynamic
instantiating app ryu.controller.ofp_handler of OFPHandler
Detected h1 MAC: 1e:35:20:8a:d0:79
Detected h2 MAC: ca:14:38:6b:90:31
Firewall: Dropping 1e:35:20:8a:d0:79 -> ca:14:38:6b:90:31
^Z
```

This starts the controller, which will listen for connections from switches.

7. Create Network Topology in Mininet:

Open a new terminal and create a simple topology with one switch and three hosts and open the new terminal and put the command:

sudo mn --topo single,3 --controller remote --switch ovsk

```
pavisha@ubuntu:~$ sudo mn --topo single,3 --controller remote --switch ovsk
[sudo] password for pavisha:
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

Here:

- single,3 → One switch with three hosts
- remote → Connects to external controller (Ryu)
- Open ovsk → Uses vSwitch kernel switch

8. Test Network Connectivity:

Test the communication between hosts using:

pingall

```
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 X
h3 -> h1 X
*** Results: 33% dropped (4/6 received)
mininet>
```

This command checks whether all hosts in the network can communicate with each other.

9. View Flow Table Entries:

Open another terminal and check the flow entries in the switch and the new terminal and put the command:

sudo ovs-ofctl dump-flows s1

10. Observe and Analyze Results:

- Verify that hosts are able to communicate initially.
- Observe how the controller dynamically learns MAC addresses.
- Check if firewall rules block specific traffic.
- Confirm that flow entries are created automatically in the switch.

RESULT:

The SDN network is successfully implemented using Mininet and controlled by the Ryu controller. The controller dynamically manages traffic, applies firewall rules, and installs flow entries in the switch.

Ex No : 4 Date :	Create a simple End-to-end Network Service with Two VNFs using vim-emu

AIM:

To create a simple end-to-end network service with two Virtual Network Functions (VNFs) using vim-emu, a tool that allows emulation of NFV scenarios using Docker containers.

PREREQUISTIES:

Ensure your system has:

- Ubuntu/Linux environment
- Internet connectivity
- Sudo privileges

PROCEDURE:

1.Install Required Packages:

Run the following commands:

```
sudo apt update
sudo apt install -y git docker-compose python3-pip
```

```
srijainthu@Jainthu-Pc:~$ sudo apt install -y git docker.io docker-compos
e python3-pip
[sudo] password for srijainthu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
The following additional packages will be installed:
  bridge-utils build-essential bzip2 containerd cpp cpp-13 cpp-13-x86-64
-linux-gnu cpp-x86-64-linux-gnu dns-root-data
  dnsmasq-base dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-
x86-64-linux-gnu gcc gcc-13 gcc-13-base
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu iptables javascript-commo
n libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 lib
atomic1 libc-dev-bin libc-devtools libc6-dev
  libcc1-0 libcrypt-dev libde265-0 libdpkg-perl libexpat1-dev libfakeroo
t libfile-fcntllock-perl libgcc-13-dev libgd3
  libgomp1 libheif-plugin-aomdec libheif-plugin-aomenc libheif-plugin-li
bde265 libheif1 libhwasan0 libip4tc2 libip6tc2
  libisl23 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan
0 libmpc3 libnetfilter-conntrack3
  libnfnetlink0 libnftables1 libnftnl11 libpython3-dev libpython3.12-dev
 libquadmath0 libstdc++-13-dev libtsan2
  libubsan1 libxpm4 linux-libc-dev lto-disabled-list make manpages-dev n
ftables pigz python3-compose python3-dev
  python3-docker python3-dockerpty python3-docopt python3-dotenv python3
-texttable python3-websocket python3-wheel
  python3.12-dev rpcsvc-proto runc ubuntu-fan zlib1g-dev
Suggested packages:
  ifupdown bzip2-doc cpp-doc gcc-13-locales cpp-13-doc aufs-tools btrfs-
progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils d
```

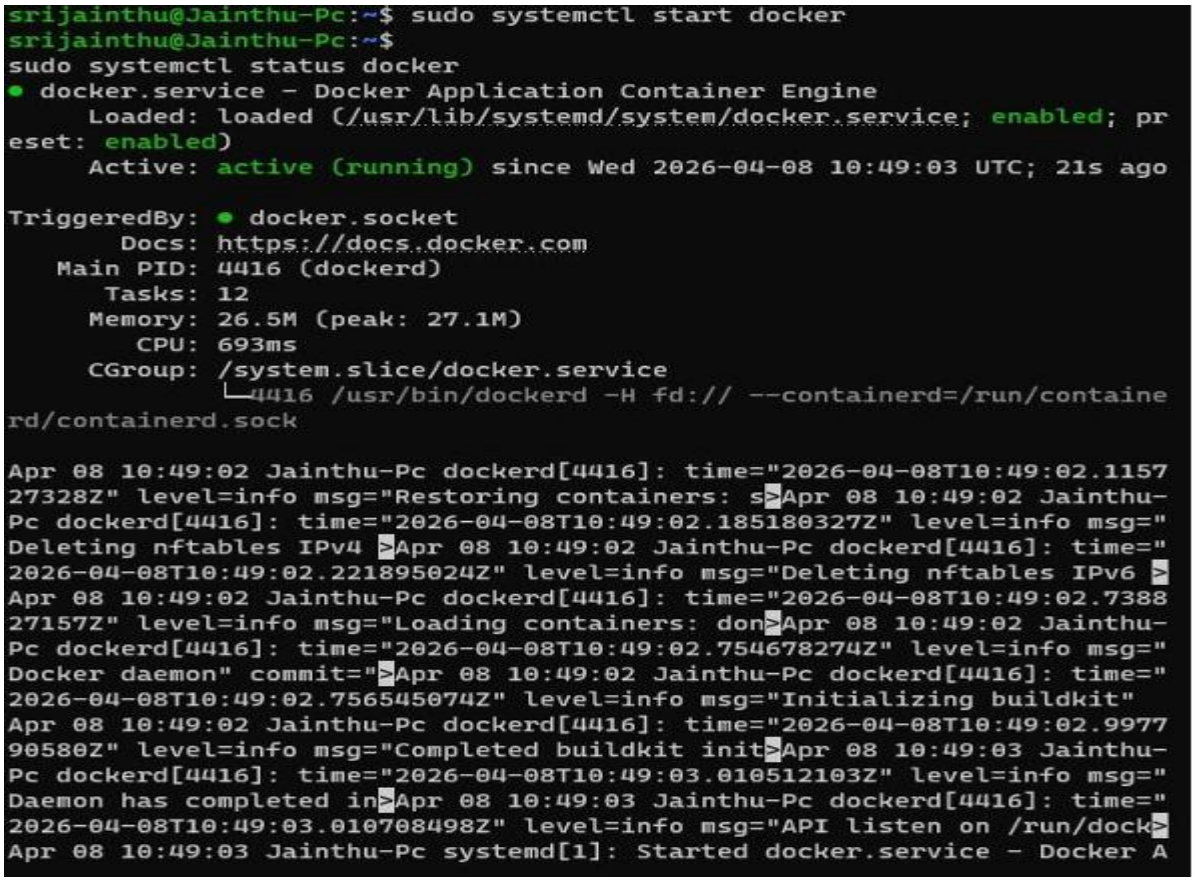
Start the Docker service:

```
sudo systemctl start docker
sudo systemctl status docker
```

(Press q to exit status view)

Verify Docker installation:

```
Docker --version
```



2.Clone vim-emu Repository:

```
git clone https://github.com/com/containernet/vim-emu.git
cd vim-emu
```

```
srijainthu@Jainthu-Pc:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.1
srijainthu@Jainthu-Pc:~$
git clone https://github.com/containernet/vim-emu.git
Cloning into 'vim-emu'...
remote: Enumerating objects: 6005, done.
remote: Counting objects: 100% (128/128), done.
remote: Compressing objects: 100% (73/73), done.
remote: Total 6005 (delta 49), reused 109 (delta 43), pack-reuse
d 5877 (from 1)
Receiving objects: 100% (6005/6005), 1.72 MiB | 728.00 KiB/s, do
ne.
Resolving deltas: 100% (3546/3546), done.
srijainthu@Jainthu-Pc:~$ cd vim-emu
srijainthu@Jainthu-Pc:~/vim-emu$ nano Dockerfile
```

3.Modify Dockerfile:

Open the Dockerfile:

```
nano Dockerfile
```

Add the following lines immediately after:

FROM containernet/containernet:latest

Add:

```
ENV LANG=C.UTF-8
ENV LC_ALL=C.UTF-8
```

Save and exit:

- Ctrl + S → Save
- Ctrl + X → Exit

4.Build Docker Image:

```
Docker build -t vim-emu-img
```

If permission error occurs:

```
sudo docker build -t vim-emu-img
```

```
srijainthu@Jainthu-Pc:~/vim-emu$
docker build -t vim-emu-img .
DEPRECATED: The legacy builder is deprecated and will be removed
 in a future release.
            Install the buildx component to build images with Bu
ildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.299MB
Step 1/26 : FROM containernet/containernet:latest
latest: Pulling from containernet/containernet
739ac7d83920: Pulling fs layer
e8ceafb6faad: Pulling fs layer
a67e12784180: Pulling fs layer
4f4fb700ef54: Pulling fs layer
e7ae86ffe2df: Pulling fs layer
5550bce5362b: Pulling fs layer
623c04987a42: Pulling fs layer
4f4fb700ef54: Pulling fs layer
f4d2ddb8b5ad: Pulling fs layer
a67e12784180: Download complete
e8ceafb6faad: Download complete
4f4fb700ef54: Download complete
f4d2ddb8b5ad: Download complete
739ac7d83920: Download complete
e7ae86ffe2df: Download complete
e7ae86ffe2df: Pull complete
e8ceafb6faad: Pull complete
5550bce5362b: Download complete
```

```
Step 20/26 : EXPOSE 9091
 ---> Running in 5e2f675a1976
 ---> Removed intermediate container 5e2f675a1976
 ---> c5e1ff783e68
Step 21/26 : EXPOSE 4000
 ---> Running in 3914852df3d5
 ---> Removed intermediate container 3914852df3d5
 ---> 2dbce5e9275a
Step 22/26 : EXPOSE 10243
 ---> Running in 4561aa18733b
 ---> Removed intermediate container 4561aa18733b
 ---> 453b062a8236
Step 23/26 : EXPOSE 9005
 ---> Running in 14948957d03b
 ---> Removed intermediate container 14948957d03b
 ---> 530207c5677d
Step 24/26 : EXPOSE 6001
 ---> Running in 392fd67fed60
 ---> Removed intermediate container 392fd67fed60
 ---> 74820bdf29ad
Step 25/26 : EXPOSE 9775
 ---> Running in 7c0abac6e850
 ---> Removed intermediate container 7c0abac6e850
 ---> 158e29891924
Step 26/26 : EXPOSE 10697
 ---> Running in 555f52a5ed38
 ---> Removed intermediate container 555f52a5ed38
 ---> fb3eea806a62
Successfully built fb3eea806a62
Successfully tagged vim-emu-img:latest
```

5.Run Docker Container:

```
sudo docker run -it --privileged \
--name vim-emu \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(pwd):/vlm-emu \
vim-emu-img bash
```

```
srijainthu@Jainthu-Pc:~/vim-emu$ sudo docker run -it --privilege
d \
--name vim-emu \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(pwd):/vlm-emu \
vim-emu-img bash
[sudo] password for srijainthu:
+ exec /containernet/util/docker/entrypoint.sh bash
 * /etc/openvswitch/conf.db does not exist
 * Creating empty database /etc/openvswitch/conf.db
 * Starting ovsdb-server
 * Configuring Open vSwitch system IDs
 * Starting ovs-vswitchd
 * Enabling remote OVSDB managers
Pulling the "ubuntu:trusty" and "ubuntu:xenial" image for later
use...
Error response from daemon: client version 1.41 is too old. Mini
mum supported API version is 1.44, please upgrade your client to
 a newer version
Error response from daemon: client version 1.41 is too old. Mini
mum supported API version is 1.44, please upgrade your client to
 a newer version
Welcome to Containernet running within a Docker container ...
root@f9a6a3dbc95b:/vim-emu# nano /vim-emu/examples/exampletopo.p
y
bash: nano: command not found
root@f9a6a3dbc95b:/vim-emu# nano /vim-emu/examples/exampletopo.p
y
```

6.Install Nano (if not available inside container):

```
apt update
apt install -y nano
```

7.Create Network Topology:

Open topology file:

nano /vim-emu/examples/exampletopo.py

Paste the following code:

```
from mininet.topo import Topo
class ExampleTopo(Topo):
        def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        s1 = self.addSwitch('s1')
       self.addLink(h1, s1)
       self.addLink(h2, s1)
topos = {
     'exampletopo': (lambda: ExampleTopo())
}
```

Save and exit:

- Ctrl + S
- Ctrl + X

8.Run the Topology:

Execute:

sudo mn --custom /vim-emu/examples/exampletopo.py --topo exampletopo

```
root@f9a6a3dbc95b:/vim-emu# sudo mn --custom /vim-emu/examples/e
xampletopo.py --topo exampletopo
*** Error setting resource limits. Mininet's performance may be
affected.
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
```

9.Verify Network:

Once containernet starts:

Check nodes:

```
nodes
```

```
containernet> nodes
available nodes are:
c0 h1 h2 s1
containernet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 10.753 seconds
root@f9a6a3dbc95b:/vim-emu#
```

10.Exit Containernet:

```
exit
```

RESULT:

Thus, a simple end-to-end network service with two VNFs was successfully created using vim-emu, and the virtual network was emulated within a Docker-based environment.

Ex No : 5 Date :	Install OSM and onboard and orchestrate network Service

AIM:

To install Open Source MANO (OSM) and orchestrate a network service using OSM by onboarding descriptors, verifying them, and instantiating them on the platform.

TOOLS & REQUIREMENTS:

- OS: Ubuntu 20.04 LTS or later
- Memory: Minimum 8 GB RAM
- CPU: Minimum 4 cores
- Software & Packages:
    - ❖ Docker
    - ❖ Snapd (for OSM installation)
    - ❖ Git
    - ❖ Python3-pip
    - ❖ Open Source MANO (OSM) client
- Internet Connectivity

THEORY:

Open Source MANO (OSM):

OSM (Open Source MANO) is an open-source project initiated and hosted by ETSI (European Telecommunications Standards Institute). It provides a full stack solution for managing and orchestrating network services in NFV (Network Functions Virtualization) environments.

It is designed to manage:

- Virtual Network Functions (VNFs)
- Network Services (NSs)
- VIMs (Virtual Infrastructure Managers) such as OpenStack, VMware, and public clouds.

Components of OSM:

Component	Description
LCM (Lifecycle Manager)	Manages the lifecycle of VNFs and NSs (creation, start, stop, scale, delete)
RO (Resource Orchestrator)	Allocates resources like compute, network, and storage from infrastructure
NBI (Northbound Interface	Exposes REST APIs and CLI for developers and administrators
VCA (VNF Configuration Agent)	Automates configuration tasks inside VNFs
MON (Monitoring Module)	Monitors VNF health and resource
OSM (osm client)	A command-line tool to interact with OSM

Onboarding in OSM:

Onboarding is the process of registering a new service or function into the OSM platform. It includes:

- Preparing the VNFD (Virtual Network Function Descriptor)
- Preparing the NSD (Network Service Descriptor)
- Uploading these descriptors into the OSM system so they can be instantiated and managed.

Orchestration:

Orchestration in the context of NFV is the automated management of:

- Deployment
- Scaling
- Configuration
- Monitoring of VNFs and NSs across different cloud environments.

OSM acts as the central brain, deciding how to deploy and manage these services based on descriptors and system status.

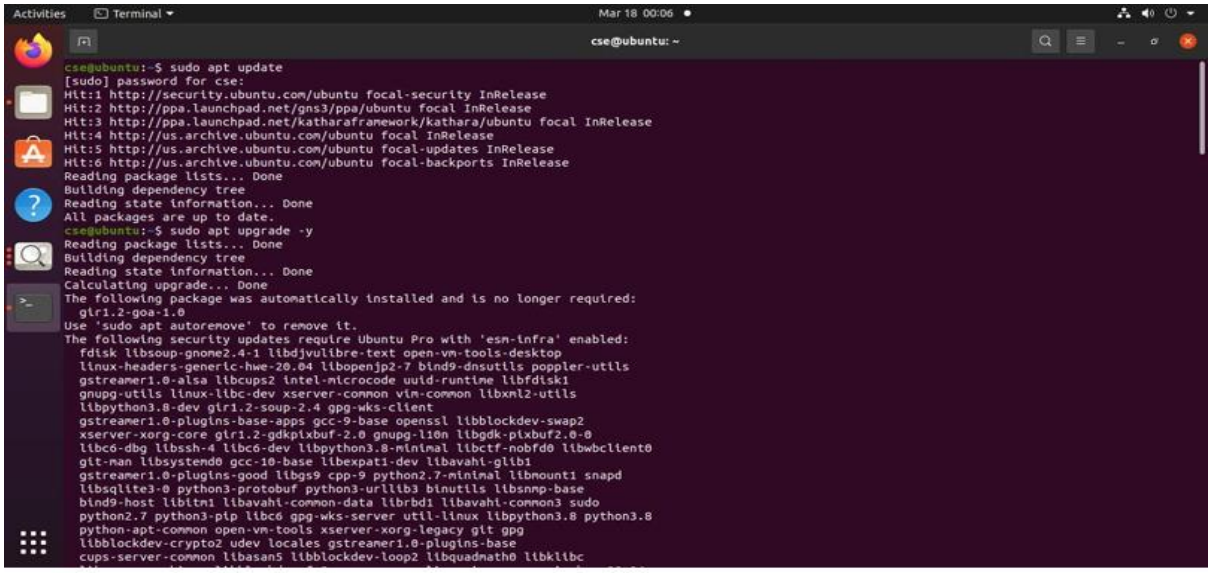
Importance of OSM in Software Defined Networks:

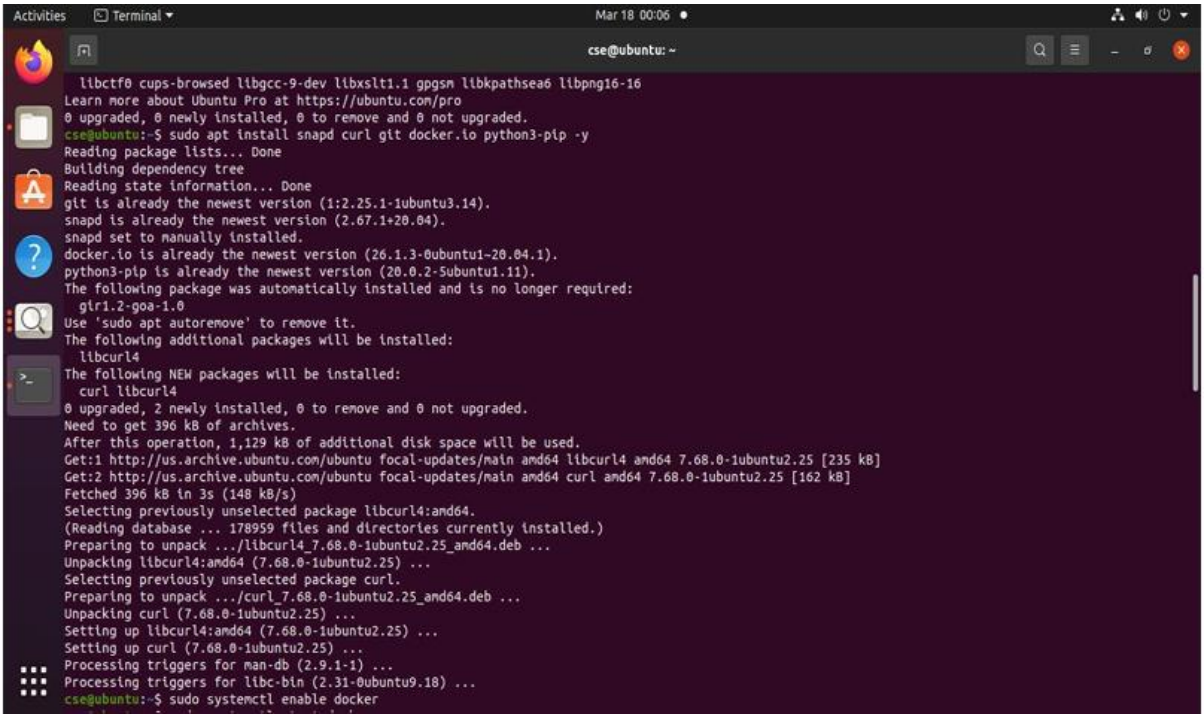
- Automates Operations – No manual configuration of VNFs
- Faster Deployment – Network services can be launched in minutes
- Scalability – VNFs can be scaled based on traffic and performance
- Cloud-Agnostic – Supports multiple infrastructures: OpenStack, AWS, VMware
- Standardized – Aligned with ETSI NFV standards

PROCEDURE:

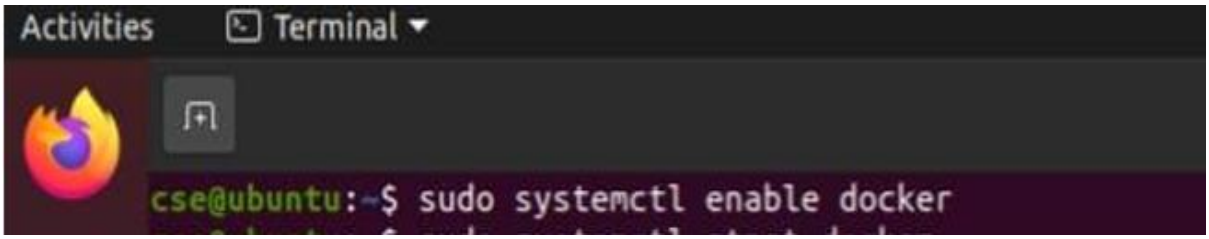
- Install necessary packages like Docker, Snap, and Git.
- Use Snap to install OSM, which includes all core modules.

sudo apt update && sudo apt upgrade -y

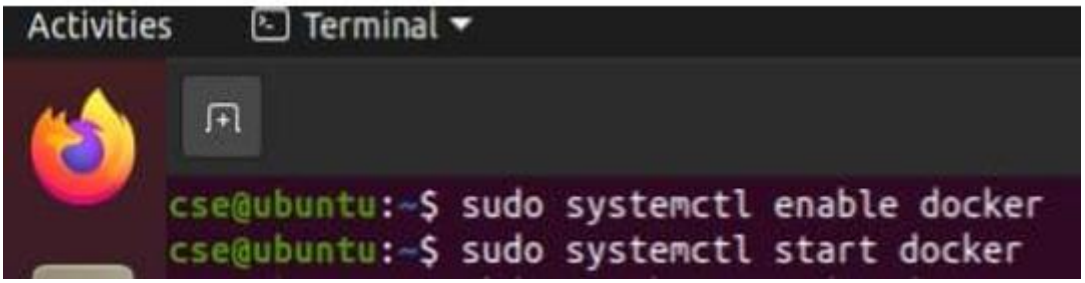




sudo systemctl enable docker

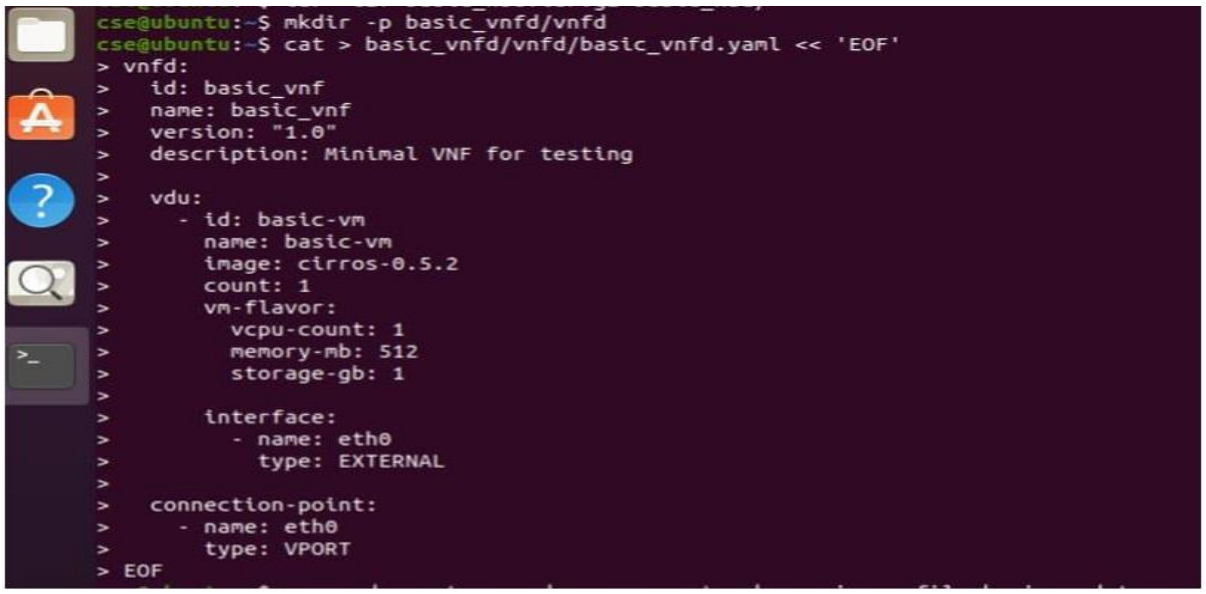


sudo systemctl start docker



Descriptor Preparation:

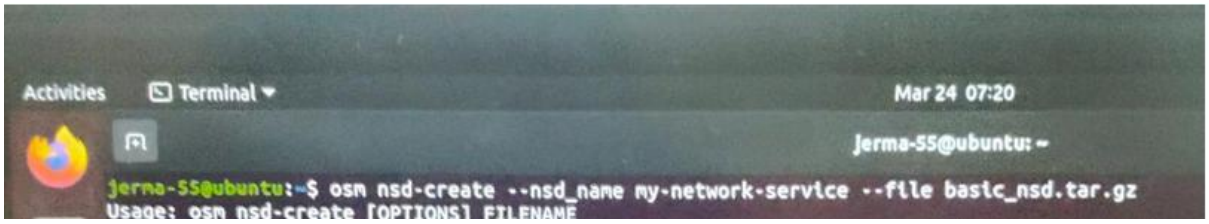
Prepare or download VNFD (Virtual Network Function and Descriptor) and NSD (Network Service Descriptor) packages. These are structured files that define how a VNF or network service behaves and is deployed.



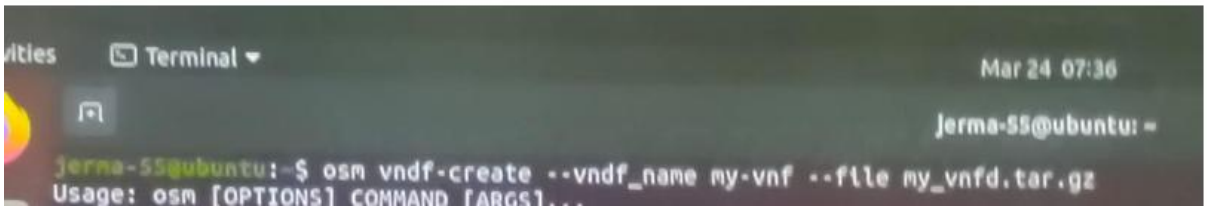
Onboarding:

- Use the osmclient to upload VNFD and NSD to the OSM platform.
- This makes the service/function available for deployment.

osm nsd-create --nsd_name my-network-service --file my_nsd.tar.gz



osm vnfd-create --vnfd_name my-vnf --file my_vnfd.tar.gz



Verification:

- Check that the descriptors have been successfully onboarded using list commands.
- You can also verify using the OSM web interface.

```
osm nsd-list
osm vnfd-list
```

Insallation:

- Deploy the network service onto the virtual infrastructure (like OpenStack).
- OSM allocates resources and launches VNFs automatically.

```
osm ns-create --nsd_name my-network-service --ns_name
my_service_instance --vim_account my_vim
```

Monitoring and Management:

- Monitor the deployed VNFs.
- If needed, scale or terminate services directly using the client interface.

```
osm ns-list
osm ns-show my_service_instance
```

Termination:

- Once testing or use is complete, services can be deleted or reconfigured.

```
osm ns-delete my_service_instance
```

RESULT:

Thus, the installation of Open Source MANO (OSM) and the onboarding and orchestration of a network service was successfully studied and understood.